

SECURE ENTERPRISE PORTAL

Employee Leave Management System

A Flask-Based Web Application for Streamlined HR Operations. Built with responsive UI, dynamic database calculations, and role-based access configurations.

DEVELOPED BY: **Dinesh K** | PROJECT DATE: **June 2026**

Introduction

⚠️ Legacy Management Problems

Manual leave tracking systems introduce severe administrative friction:

- ✖ **Calculation Liabilities:** Manual leave balance calculations are highly error-prone.
- ✖ **Slow Processing:** Approvals through paper forms or loose emails waste valuable time.
- ✖ **Zero Visibility:** Lack of real-time balance metrics reduces operational transparency.

✦ The Modern Automated Purpose

To completely digitize, centralize, and automate employee leave workflows:

- ✔ **Accurate Calculations:** Database-driven balance logs prevent manual math mistakes.
- ✔ **High-Efficiency:** Instant communication links managers directly with employees.
- ✔ **Transparent Dashboarding:** Provides complete view of remaining allowances.

Project Objectives

🎯 Scope & Goals

The system is designed to eliminate legacy administrative overhead by building direct integration pipelines between employees and HR administrators, allowing transaction-safe balance execution under a modern interface.

- 🎯 **Automate Workflows:** Digitize leave application and HR manager approval systems.
- 🎯 **Real-Time Audits:** Track sick, casual, and earned balances automatically.
- 🎯 **Onboarding Gateway:** Let administrators verify and manage registrations securely.
- 🎯 **Domain Restricting:** Force email registrations to corporate domains strictly.
- 🎯 **Universal Responsive UI:** Provide structured navigation scaling onto mobile viewports.

Core Technology Stack



Backend

Python 3.8+ combined with Python Flask 3.0 frameworks.



Database

SQLite engine with declarative models via Flask-SQLAlchemy.



Security

Werkzeug password hashing and secure dynamic cookie sessions.



Deployment

Gunicorn WSGI web server live-hosted on Render.com.

System Architecture Pattern

MVC Paradigm Pattern

Separates database schemas, front-end visual states, and backend application routes:

-  **Models:** Declarative database tables structured via Flask-SQLAlchemy.
-  **Templates:** Renders secure, dynamic user dashboards with Jinja2 syntax.
-  **Routes:** Clean controller logic mappings defined inside the core app.py script.

 **Browser Front-End**



HTML5 / CSS3 / Vanilla JS

 **Flask App (app.py)**



Business Logic Core

 **SQLite Database**



Relational Tables Storage

Relational Database Design

TABLE NAME	SCHEMA COLUMN MAPPING DETAILS	ROLE & CORE STRUCTURAL LOGIC
Settings	id, key (String), value (Text)	Stores setup parameters like the company name and approved registration domain limits.
Admins	id, name, email, password (PBKDF2 Hash)	Secures privileged access credentials for handling admin and management features.
Employees	id, emp_id, allowance_balances, status	Tracks detailed metadata alongside active leave banks (Casual: 12, Sick: 10, Earned: 15).
LeaveRequests	id, employee_id (FK), dates, status, remarks	Maintains historical records and review status tags (Pending / Approved / Rejected).

Portal Feature Matrix

Employee Capabilities

- ✓ **Accrual Bars:** Visually tracking remaining balances with styled trackers.
- ✓ **Validation Forms:** Dynamic days counter preventing manual submission mistakes.
- ✓ **Audit History:** Real-time logging of status tags to monitor progression transparently.

Administrative Controls

- ✓ **Setup Wizard:** First-run database seeding to set domain restrictions.
- ✓ **User Verification:** Gatekeeping registry access to protect company assets.
- ✓ **Evaluation Inbox:** Quick-action workspace to review and process applications.

Security & Structural Safeguards

Key System Safeguards

Active validation rules protect application routes and backend database assets:

-  **Hashed Passwords:** PBKDF2 hash structures secure sensitive login details.
-  **Database Parameters:** Parameterized queries block SQL injection risks.
-  **Decorators & Middleware:** Session check layers protect admin workspaces.
-  **Closed Domain Gate:** Restricts registration to corporate domains.



Zero-Trust Processing

Checks user permission layers on every server request, blocking unauthorized endpoint execution.

Deployment & Future Vision

Render Deployment

- ▶ **WSGI Server:** Powered by Gunicorn engine layers.
- ▶ **CI/CD Pipelines:** Direct continuous deployment hooks.
- ▶ **Active URL:** `employee-leave-management-an6e.onrender.com`

Upcoming Features

- ▶ **Email Triggers:** SMTP automatic mail logs upon approval.
- ▶ **Scalable Storage:** Moving from SQLite to PostgreSQL tiers.
- ▶ **Visual Calendars:** Interactive holiday planning grid timelines.

STREAMLINED HR WORKFLOWS

Employee Leave System

A clean, secure, and production-ready solution built to automate manual workflows, prevent human error, and improve team coordination.

GITHUB SOURCE CODE REPOSITORY

 github.com/dineshTech-creator/Employee_Leave_management